

Mục lục

1. Thực hành Context Dependency Injection (CDI)	1
1. Những vấn đề khi chỉ dùng Java Servlet thuần	1
2. Vì sao nên dùng CDI trong Servlet?	1
Giảm khớp nối với @Inject	1
Quản lý Scope theo ngữ cảnh tự động (Contextual Lifecycle)	2
Tách biệt mối quan tâm (AOP với Interceptors và Decorators)	2
Xử lý sự kiện linh hoạt (Event & Observer)	2
3. So sánh Servlet thuần và Servlet có CDI	2
1. Phân biệt Bean và Servlet Class	3
2. Ví dụ trực quan so sánh cấu trúc	4
3. Tóm tắt mối quan hệ	5
4. Đặc tính bắt buộc của một JavaBean	5
5. Ví dụ code JavaBean chuẩn	5
6. Ứng dụng thực tế của JavaBean	7
4. Cấu hình dự án CDI trong IntelliJ IDEA , bạn thực hiện 2 bước sau:	7
Bước 1: Thêm Dependency vào file pom.xml	7
Bước 2: Reload lại Maven trong IntelliJ	8
5. Thực hành: Dự án ShoppingCart sử dụng Context Dependency Injection (CDI)	8
5.1. Cấu hình với Tomcat	10
pom.xml	10
beans.xml	11
web.xml	11
Source code	12
2. Thực hành: REST API	23
1. REST API là gì?	23
2. Tại sao cần dùng REST API?	24
3. REST API và RESTful API	25
Dự án Shopping Cart	26
BackEnd: Jakarta EE: RESTful API	26
1. Kiểm tra plugin IntelliJ	26
2. Chuẩn bị JDK và Tomcat	27
3. Tạo Jakarta EE REST Service project	27
4. Chọn Jakarta EE version và dependencies	27
5. Kiểm tra cấu trúc được tạo	28
6. Chỉnh thông tin Maven (file pom.xml)	28
7. Khai báo dependency cho Tomcat(file pom.xml)	28
8. Tạo các package	30
9. Kích hoạt CDI bằng beans.xml	31
10. Tạo web.xml	32
11. Tạo Jakarta REST Application	32
12. Tạo model Product	32
13. Tạo model CartItem	34
14. Tạo CDI service ProductCatalog	34

15. Tạo AuthenticationService	35
16. Tạo ShoppingCart	36
17. Tạo UserSession @SessionScoped	37
18. Tạo AuthResource	38
19. Tạo ProductResource	40
20. Tạo CartResource	41
21. Tạo Tomcat configuration	42
22. Build và chạy	43
23. Tạo IntelliJ HTTP Client file	43
24. Kết quả mong đợi	44
FrontEnd: WEB app (các trang JSP gọi BackEnd: các REST APIs)	45
Login.jsp	48
Products.jsp	49

1. Thực hành: Context Dependency Injection (CDI)

Trong lập trình ứng dụng Java Web truyền thống, **Servlet** đảm nhận vai trò tiếp nhận và xử lý HTTP Request. Tuy nhiên, khi ứng dụng bắt đầu phức tạp, việc chỉ dùng Servlet thuần sẽ bộc lộ nhiều điểm hạn chế.

CDI (Contexts and Dependency Injection) là chuẩn của Jakarta EE / Java EE nhằm giải quyết các vấn đề về **quản lý vòng đời đối tượng** và **tiêm phụ thuộc (Dependency Injection)**.

1. Những vấn đề khi chỉ dùng Java Servlet thuần

Khi không có CDI, bạn thường phải đối mặt với các vấn đề sau:

- **Khớp nối cứng (Tight Coupling):** Bạn phải tự tay khởi tạo đối tượng phụ thuộc (`new UserService()`, `new DatabaseConnection()`). Nếu thay đổi cấu trúc hoặc lớp triển khai, bạn phải sửa mã nguồn ở tất cả các Servlet liên quan.
-
- **Quản lý phạm vi (Scope) thủ công, dễ lỗi:** Để chia sẻ dữ liệu theo phạm vi Request, Session hay Application, bạn phải dùng các hàm:
-

- **Java**

```
HttpSession session = request.getSession();
```

```
User user = (User) session.getAttribute("currentUser"); // Dễ NullPointerException, lỗi ép kiểu (type-unsafe)
```

-

- **Khó kiểm thử (Unit Testing):** Các Servlet phụ thuộc cứng vào `HttpServletRequest`, `HttpServletResponse` và các đối tượng `new` trực tiếp khiến việc viết Unit Test cô lập logic nghiệp vụ cực kỳ khó khăn.
-
- **Trùng lặp mã (Boilerplate Code):** Code khởi tạo kết nối DB, ghi log, phân quyền... bị lặp lại ở nhiều phương thức `doGet()` và `doPost()`.
-

2. Vì sao nên dùng CDI trong Servlet?

Giảm khớp nối với @Inject

Thay vì tự khởi tạo đối tượng, CDI container sẽ tự động tìm, khởi tạo và đưa (inject) đối tượng cần thiết vào Servlet.

Java

```
@WebServlet("/profile")
public class UserServicelet extends HttpServlet {

    @Inject
    private UserService userService; // CDI tự động tiêm đối tượng phù hợp

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) {
        resp.getWriter().write(userService.getUserInfo());
    }
}
```

Quản lý Scope theo ngữ cảnh tự động (Contextual Lifecycle)

CDI tự động gắn kết vòng đời của các Bean với vòng đời của HTTP Request/Session thông qua các Annotation:

- **@RequestScoped:** Đối tượng chỉ tồn tại trong vòng đời của 1 HTTP Request.
-
- **@SessionScoped:** Đối tượng tồn tại gắn liền với phiên làm việc (Session) của người dùng.
-
- **@ApplicationScoped:** Đối tượng Singleton dùng chung cho toàn ứng dụng.
-

Bạn không cần tự lưu hay lấy dữ liệu từ `session.getAttribute()/setAttribute()` bằng chuỗi ký tự thủ công.

Tách biệt mối quan tâm (AOP với Interceptors và Decorators)

CDI cho phép chặn các lời gọi phương thức để xử lý logic chung (Ghi log, kiểm tra quyền hạn, quản lý Transaction, bắt lỗi) bằng `@Interceptor` mà không làm bẩn code xử lý chính của Servlet/Service.

Xử lý sự kiện linh hoạt (Event & Observer)

CDI cung cấp cơ chế Event giúp các thành phần giao tiếp với nhau mà không cần biết nhau:

- Servlet phát sự kiện: `event.fire(new UserLoggedInEvent(user));`
-
- Service lắng nghe: `public void onUserLogin(@Observes UserLoggedInEvent event) { ... }`
-

3. So sánh Servlet thuần và Servlet có CDI

Tiêu chí	Servlet thuần	Servlet kết hợp CDI
Khởi tạo đối tượng	Tự gọi <code>new Service()</code>	Container tự tiêm qua <code>@Inject</code>
Lưu trữ Session / Request	Thủ công (<code>setAttribute</code> , <code>getAttribute</code>)	Tự động qua <code>@SessionScoped</code> , <code>@RequestScoped</code>
Khả năng Test	Khó Mock/Stub các phụ thuộc	Dễ dàng Mock các Bean được inject
Kiến trúc mã nguồn	Dễ phình to, trộn lẫn logic điều hướng và nghiệp vụ	Phân tách rõ ràng (Clean Architecture, SoC)
Mở rộng logic chéo (Logging, Tx)	Viết thủ công ở từng Servlet hoặc Filter	Sử dụng CDI Interceptor

Tóm lại: CDI giúp Servlet trở về đúng vai trò nguyên bản của nó: **tầng điều hướng (Controller)**. Mọi logic khởi tạo, quản lý vòng đời và xử lý dữ liệu được chuyển giao cho CDI Container quản lý, giúp mã nguồn an toàn kiểu dữ liệu (type-safe), dễ bảo trì và mở rộng.

Bean (trong ngữ cảnh CDI/Jakarta EE) là một class Java thông thường (POJO-Plain Old Java Object (Đối tượng Java thuần túy cũ)) được **CDI Container** quản lý toàn bộ vòng đời (khởi tạo, tiêm phụ thuộc, hủy) và gắn liền với một phạm vi ngữ cảnh cụ thể (Request, Session, Application).

1. Phân biệt Bean và Servlet Class

Tiêu chí	Servlet Class	CDI Bean
Bản chất	Kế thừa từ <code>HttpServlet</code> . Là điểm tiếp nhận HTTP request trực tiếp từ client.	Class Java thuần túy (POJO), thường không kế thừa class đặc thù nào.
Đơn vị quản lý	Servlet Container (Tomcat, Catalina, Jetty...).	CDI Container (Weld, OpenWebBeans...).
Nhiệm vụ chính	Điều hướng (Routing): Nhận HTTP Request (GET, POST), đọc header/param, gọi tầng logic và trả về HTTP Response (HTML, JSON, Forward JSP).	Nghịệp vụ (Business Logic): Tính toán, truy vấn dữ liệu, xử lý logic hệ thống, lưu trạng thái dữ liệu (State).
Vòng đời (Lifecycle)	Thường là Singleton (1 instance duy nhất phục vụ mọi request bằng đa luồng đa luồng - multithreading).	Linh hoạt tùy theo Scope: <code>@RequestScoped</code> , <code>@SessionScoped</code> , <code>@ApplicationScoped</code> ...
Khả năng tái sử dụng	Rất thấp, bị gắn cứng với môi trường Web/HTTP API.	Rất cao, có thể tái sử dụng ở bất kỳ đâu (Console app, REST API, JMS, Cron job...).
Khả năng kiểm thử (Testing)	Khó Unit Test độc lập vì dính lúu đến <code>HttpServletRequest</code> , <code>HttpServletResponse</code> .	Cực kỳ dễ viết Unit Test vì chỉ nhận dữ liệu thuần qua tham số phương thức.

2. Ví dụ trực quan so sánh cấu trúc

Servlet Class (Tầng Giao tiếp HTTP):

Java

```
@WebServlet("/order")
public class OrderServlet extends HttpServlet {
```

```
@Inject
```

```
private OrderService orderService; // Tiêm Bean vào để sử dụng
```

```

@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp) {
    String productId = req.getParameter("id");
    // Servlet chỉ điều phối, chuyển việc tính toán cho Bean
    orderService.placeOrder(productId);
}
}

```

CDI Bean (Tăng Nghiệp vụ/Dữ liệu):

Java

```

@ApplicationScoped
public class OrderService { // Là một POJO Bean thuần túy, không dính dáng HttpServlet

    @Inject
    private PaymentService paymentService; // Bean này có thể inject Bean khác

    public void placeOrder(String productId) {
        // Xử lý logic nghiệp vụ, trừ kho, gọi thanh toán...
        paymentService.pay(productId);
    }
}

```

3. Tóm tắt mối quan hệ

- **Servlet** đóng vai trò là "người bảo vệ ở cổng": tiếp nhận các gói tin HTTP gửi đến từ trình duyệt.
- **Bean** đóng vai trò là "các chuyên gia bên trong": giải quyết các yêu cầu tính toán, lưu trữ và logic nghiệp vụ.
- Servlet không tự giải quyết hết mọi việc mà **sử dụng (inject) các Bean** để thực hiện logic nghiệp vụ rồi nhận kết quả trả về cho client.

JavaBean Component (thường gọi tắt là **JavaBean**) là một chuẩn quy ước (convention) trong Java cho việc xây dựng các class có thể tái sử dụng, đóng gói nhiều thuộc tính thành một đối tượng duy nhất để các framework hoặc công cụ khác có thể thao tác tự động thông qua cơ chế phản chiếu (Reflection).

4 Đặc tính bắt buộc của một JavaBean

Để một class Java thông thường trở thành một JavaBean chuẩn, nó phải tuân thủ nghiêm ngặt 4 quy tắc sau:

1. **Có Constructor không tham số (No-argument Constructor):** Bắt buộc phải có constructor mặc định public để các framework/container có thể khởi tạo đối tượng tự động qua `Class.forName().getDeclaredConstructor().newInstance()`.
- 2.
3. **Các thuộc tính (Properties) phải là private:** Đảm bảo tính đóng gói (Encapsulation), không cho phép truy cập trực tiếp từ bên ngoài.
- 4.
5. **Cung cấp cặp phương thức Getter/Setter chuẩn public:**

6.

- Lấy giá trị: `get<PropertyName>()` (hoặc `is<PropertyName>()` đối với kiểu `boolean`).
-
- Gán giá trị: `set<PropertyName>(...)`.
-

7. **Implement interface `java.io.Serializable`**: Cho phép chuyển đổi trạng thái của đối tượng thành luồng byte để lưu trữ vào file, truyền qua mạng, hoặc lưu vào `HttpSession`.

8.

5. Ví dụ code JavaBean chuẩn

Java

```
package com.example.model;
```

```
import java.io.Serializable;
```

```
// 1. Implement Serializable
```

```
public class UserBean implements Serializable {
```

```
    private static final long serialVersionUID = 1L;
```

```
// 2. Thuộc tính private
```

```
    private String username;
```

```
    private int age;
```

```
    private boolean active;
```

```
// 3. Constructor không tham số (public)
```

```
    public UserBean() {
```

```
    }
```

```
// Constructor có tham số (tùy chọn, phục vụ việc code thủ công)
```

```
    public UserBean(String username, int age, boolean active) {
```

```
        this.username = username;
```

```
        this.age = age;
```

```
        this.active = active;
```

```
    }
```

```
// 4. Getter và Setter tuân thủ chuẩn đặt tên
```

```
    public String getUsername() {
```

```
        return username;
```

```
    }
```

```
    public void setUsername(String username) {
```

```
        this.username = username;
```

```
    }
```

```
    public int getAge() {
```

```
        return age;
```

```

    }

    public void setAge(int age) {
        this.age = age;
    }

    public boolean isActive() { // Kiểu boolean dùng tiền tố 'is'
        return active;
    }

    public void setActive(boolean active) {
        this.active = active;
    }
}

```

6. Ứng dụng thực tế của JavaBean

- **DTO / VO (Data Transfer Object / Value Object):** Dùng để đóng gói dữ liệu truyền tải giữa các tầng trong ứng dụng (ví dụ: chuyển dữ liệu từ Database → Service → Servlet → JSP).
- **Binding dữ liệu trong JSP / JSF:** Trang JSP có thể tự động ánh xạ dữ liệu từ form vào JavaBean thông qua thẻ `<jsp:useBean>` hoặc đọc dữ liệu hiển thị bằng Expression Language (EL):

- **Java**

```

<!-- Truy xuất tự động qua getter getUsername() -->
<p>Tên người dùng: ${user.username}</p>

```

- **Anh xạ cơ sở dữ liệu (ORM - Hibernate, JPA, MyBatis):** Các ORM framework yêu cầu các Entity phải tuân theo chuẩn JavaBean để tự động đọc/ghi dữ liệu từ các cột trong Database vào thuộc tính class.
- **Tuần tự hóa JSON/XML (Jackson, Gson):** Các thư viện chuyển đổi dữ liệu dựa vào Getter/Setter của JavaBean để serialize/deserialize dữ liệu JSON khi xây dựng REST API.

Lỗi chữ đỏ (`jakarta.enterprise` và `jakarta.inject`) xảy ra do dự án Maven của bạn chưa khai báo dependency chứa API của **Jakarta CDI** và **Jakarta Inject** trong file `pom.xml`.

4. Cấu hình dự án CDI trong IntelliJ IDEA , bạn thực hiện 2 bước sau:

Bước 1: Thêm Dependency vào file `pom.xml`

Mở file `pom.xml` (`CDIDemo`) (tab đầu tiên bên góc trái) và thêm các dependency sau vào bên trong thẻ `<dependencies>...</dependencies>`:

XML

```

<dependencies>

```



```

<!-- Jakarta Enterprise Beans / CDI API -->
<dependency>
  <groupId>jakarta.enterprise</groupId>
  <artifactId>jakarta.enterprise.cdi-api</artifactId>
  <version>4.0.1</version>
  <scope>provided</scope>
</dependency>

<!-- Jakarta Inject API (cung cấp @Named, @Inject) -->
<dependency>
  <groupId>jakarta.inject</groupId>
  <artifactId>jakarta.inject-api</artifactId>
  <version>2.0.1</version>
  <scope>provided</scope>
</dependency>
</dependencies>

```

Lưu ý về <scope>:

- Nếu bạn deploy lên full server như **WildFly, GlassFish, Payara, TomEE**, giữ nguyên <scope>provided</scope>.
-
- Nếu bạn chạy trên **Apache Tomcat** thuần (Tomcat 10+), hãy bỏ dòng <scope>provided</scope> hoặc bổ sung implementation **Weld Servlet** để CDI có thể chạy được runtime:
-

• XML

```

<dependency>
  <groupId>org.jboss.weld.servlet</groupId>
  <artifactId>weld-servlet-core</artifactId>
  <version>5.1.2.Final</version>
</dependency>

```

•

Bước 2: Reload lại Maven trong IntelliJ

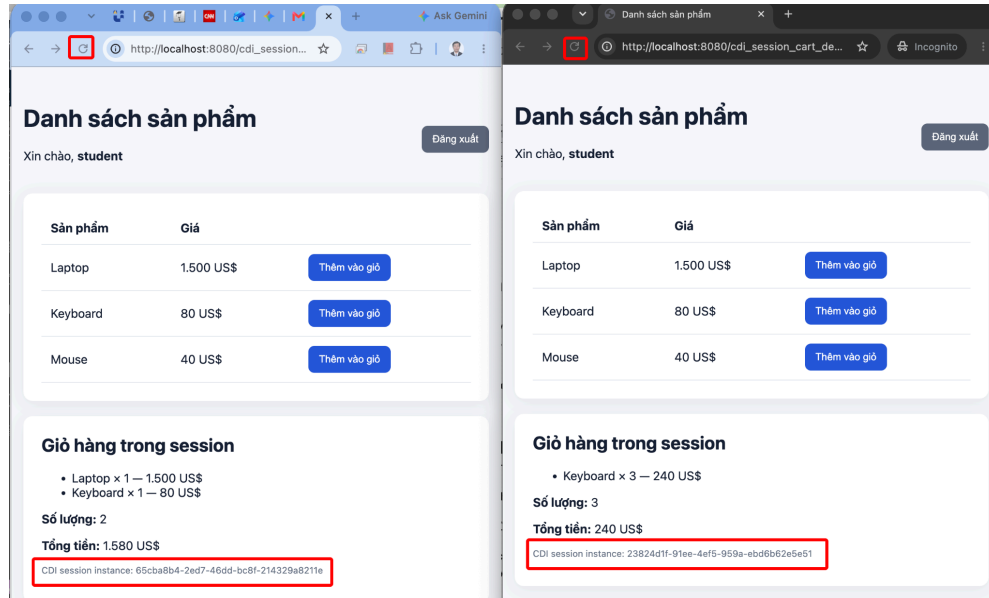
Sau khi lưu file pom.xml:

1. Nhấn vào biểu tượng **con voi/chữ M nhỏ** (Reload All Maven Projects) xuất hiện ở góc trên bên phải màn hình chỉnh sửa code.
- 2.
3. Hoặc mở thanh công cụ **Maven** ở cạnh phải màn hình → nhấn nút **Reload** (biểu tượng 2 mũi tên xoay tròn 0).
- 4.

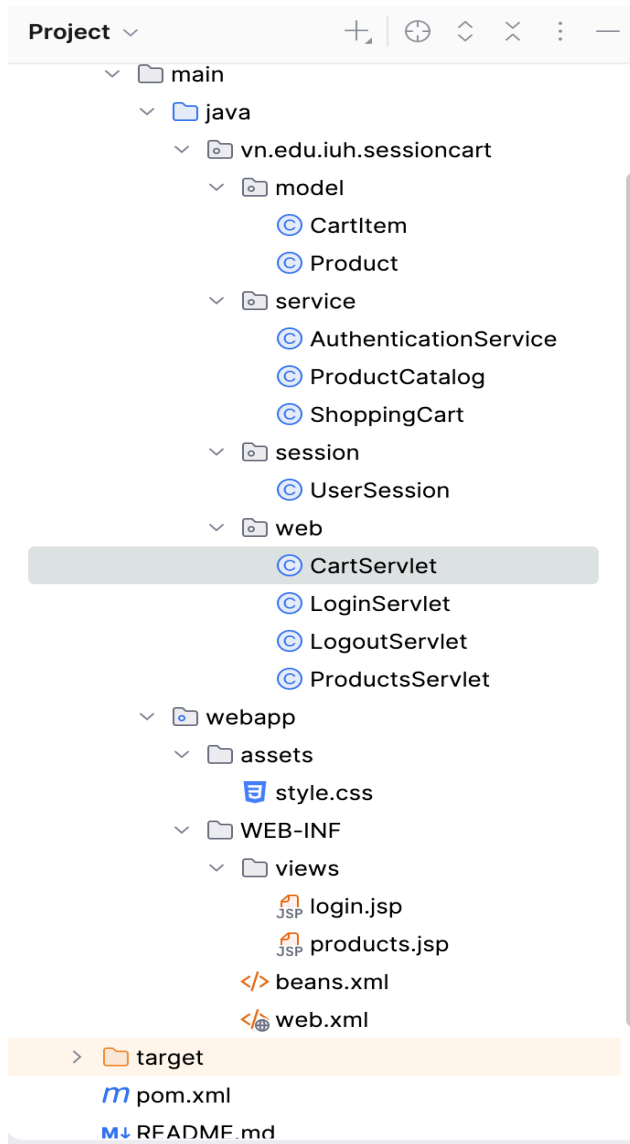
Sau khi IntelliJ tải xong thư viện, các dòng import `jakarta.enterprise...` và `jakarta.inject...` sẽ hết báo đỏ ngay lập tức.

5. Thực hành: Dự án ShoppingCart sử dụng Context Dependency Injection (CDI)

Yêu cầu: Tạo dự án quản lý giỏ hàng (ShoppingCart) sao cho mỗi giao dịch của 1 người dùng đăng nhập để mua hàng chỉ có hiệu lực trong giao dịch đó.



Hình 1- Hai cửa sổ ShoppingCart ở 2 giao dịch khác nhau



Hình 2- Cấu trúc dự án

5.1. Cấu hình với Tomcat

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>vn.edu.iuh</groupId>
  <artifactId>cdi-session-cart-demo</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>war</packaging>
```

```

<properties>
  <maven.compiler.release>21</maven.compiler.release>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
  <!-- Tomcat cung cấp Servlet API. -->
  <dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.1.0</version>
    <scope>provided</scope>
  </dependency>

  <!-- Tomcat không có CDI, vì vậy đóng gói Weld vào WAR.
-->
  <dependency>
    <groupId>org.jboss.weld.servlet</groupId>
    <artifactId>weld-servlet-core</artifactId>
    <version>5.1.2.Final</version>
  </dependency>

  <!-- JSTL API và implementation cho JSP. -->
  <dependency>
    <groupId>jakarta.servlet.jsp.jstl</groupId>
    <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
    <version>3.0.2</version>
  </dependency>
  <dependency>
    <groupId>org.glassfish.web</groupId>
    <artifactId>jakarta.servlet.jsp.jstl</artifactId>
    <version>3.0.1</version>
  </dependency>
</dependencies>

<build>
  <finalName>cdi-session-cart-demo</finalName>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.13.0</version>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>

```

```

        <artifactId>maven-war-plugin</artifactId>
        <version>3.4.0</version>
        <configuration>

<failOnMissingWebXml>false</failOnMissingWebXml>
        </configuration>
    </plugin>
</plugins>
</build>
</project>

```

beans.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="https://jakarta.ee/xml/ns/jakartaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/beans_4_0.xsd"
    version="4.0"
    bean-discovery-mode="annotated"/>

```

web.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
    version="6.0">
    <display-name>CDI Session Cart Demo</display-name>
    <session-config>
        <session-timeout>20</session-timeout>
    </session-config>
    <welcome-file-list>
        <welcome-file>login</welcome-file>
    </welcome-file-list>
</web-app>

```

Source code

```

package vn.edu.iuh.sessioncart.model;

import java.io.Serial;
import java.io.Serializable;
import java.math.BigDecimal;

```

```

public class Product implements Serializable {
    @Serial
    private static final long erialVersionUID = 1L;

    private final long id;
    private final String name;
    private final BigDecimal price;

    public Product(long id, String name, BigDecimal price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    public long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public BigDecimal getPrice() {
        return price;
    }
}

```

```

package vn.edu.iuh.sessioncart.model;

```

```

import java.io.Serial;
import java.io.Serializable;
import java.math.BigDecimal;

```

```

public class CartItem implements Serializable {
    @Serial
    private static final long serialVersionUID = 1L;

    private final Product product;
    private int quantity;

    public CartItem(Product product) {
        this.product = product;
        this.quantity = 1;
    }
}

```

```

    public void increaseQuantity() {
        quantity++;
    }

    public Product getProduct() {
        return product;
    }

    public int getQuantity() {
        return quantity;
    }

    public BigDecimal getSubtotal() {
        return
product.getPrice().multiply(BigDecimal.valueOf(quantity));
    }
}

package vn.edu.iuh.sessioncart.service;

import jakarta.enterprise.context.ApplicationScoped;

/** Business service minh họa. Trong hệ thống thật, dữ liệu đến
từ DB và mật khẩu phải được hash. */
@ApplicationScoped
public class AuthenticationService {
    public boolean authenticate(String username, String password) {
        return "student".equals(username) &&
"123456".equals(password);
    }
}

package vn.edu.iuh.sessioncart.service;

import jakarta.enterprise.context.ApplicationScoped;
import vn.edu.iuh.sessioncart.model.Product;

import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;

@ApplicationScoped
public class ProductCatalog {
    private final List<Product> products = List.of(
        new Product(1, "Laptop", new BigDecimal("1500.00")),

```

```

        new Product(2, "Keyboard", new BigDecimal("80.00")),
        new Product(3, "Mouse", new BigDecimal("40.00"))
    );

    public List<Product> findAll() {
        return products;
    }

    public Optional<Product> findById(long id) {
        return products.stream().filter(product -> product.getId()
== id).findFirst();
    }
}

package vn.edu.iuh.sessioncart.service;

import vn.edu.iuh.sessioncart.model.CartItem;
import vn.edu.iuh.sessioncart.model.Product;

import java.io.Serial;
import java.io.Serializable;
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** Business object thuộc về đúng một người dùng/session. */
public class ShoppingCart implements Serializable {
    @Serial
    private static final long serialVersionUID = 1L;

    private final List<CartItem> items = new ArrayList<>();

    public void add(Product product) {
        items.stream()
            .filter(item -> item.getProduct().getId() ==
product.getId())
            .findFirst()
            .ifPresentOrElse(CartItem::increaseQuantity, () ->
items.add(new CartItem(product)));
    }

    public List<CartItem> getItems() {
        return Collections.unmodifiableList(items);
    }
}

```



```

    public int getItemCount() {
        return
items.stream().mapToInt(CartItem::getQuantity).sum();
    }

    public BigDecimal getTotal() {
        return items.stream()
            .map(CartItem::getSubtotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }
}

package vn.edu.iuh.sessioncart.session;

import jakarta.enterprise.context.SessionScoped;
import jakarta.inject.Named;
import vn.edu.iuh.sessioncart.service.ShoppingCart;

import java.io.Serial;
import java.io.Serializable;
import java.util.UUID;

/**
 * CDI tạo một instance cho mỗi HTTP session.
 * sessionId dùng để quan sát instance không đổi qua nhiều
 * request.
 */
@Named("userSession")
@SessionScoped
public class UserSession implements Serializable {
    @Serial
    private static final long serialVersionUID = 1L;

    private final String instanceId = UUID.randomUUID().toString();
    private final ShoppingCart shoppingCart = new ShoppingCart();
    private String username;

    public void login(String username) {
        this.username = username;
    }

    public boolean isLoggedIn() {
        return username != null;
    }
}

```

```

    public String getUsername() {
        return username;
    }

    public ShoppingCart getShoppingCart() {
        return shoppingCart;
    }

    public String getInstanceId() {
        return instanceId;
    }
}

package vn.edu.iuh.sessioncart.web;

import jakarta.inject.Inject;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import vn.edu.iuh.sessioncart.service.AuthenticationService;
import vn.edu.iuh.sessioncart.session.UserSession;

import java.io.IOException;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    @Inject
    private AuthenticationService authenticationService;

    @Inject
    private UserSession userSession;

    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        if (userSession.isLoggedIn()) {
            response.sendRedirect(request.getContextPath() +
                "/products");
            return;
        }

        request.getRequestDispatcher("/WEB-INF/views/login.jsp").forward(r
            equest, response);
    }
}

```

```

    }

    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        request.setCharacterEncoding("UTF-8");
        String username = request.getParameter("username");
        String password = request.getParameter("password");

        if (!authenticationService.authenticate(username,
password)) {
            request.setAttribute("error", "Tên đăng nhập hoặc mật
khẩu không đúng.");
            request.setAttribute("username", username);

            request.getRequestDispatcher("/WEB-INF/views/login.jsp").forward(r
equest, response);
            return;
        }

        userSession.login(username);
        response.sendRedirect(request.getContextPath() +
"/products");
    }
}

package vn.edu.iuh.sessioncart.web;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws IOException {
        if (request.getSession(false) != null) {
            request.getSession(false).invalidate();
        }
        response.sendRedirect(request.getContextPath() + "/login");
    }
}

```

```

}

package vn.edu.iuh.sessioncart.web;

import jakarta.inject.Inject;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import vn.edu.iuh.sessioncart.service.ProductCatalog;
import vn.edu.iuh.sessioncart.session.UserSession;

import java.io.IOException;

@WebServlet("/products")
public class ProductsServlet extends HttpServlet {
    @Inject
    private ProductCatalog productCatalog;

    @Inject
    private UserSession userSession;

    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        if (!userSession.isLoggedIn()) {
            response.sendRedirect(request.getContextPath() +
                "/login");
            return;
        }

        request.setAttribute("products", productCatalog.findAll());
        request.setAttribute("userSession", userSession);

        request.getRequestDispatcher("/WEB-INF/views/products.jsp").forward(
            request, response);
    }
}

package vn.edu.iuh.sessioncart.web;

import jakarta.inject.Inject;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;

```

```

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import vn.edu.iuh.sessioncart.model.Product;
import vn.edu.iuh.sessioncart.service.ProductCatalog;
import vn.edu.iuh.sessioncart.service.ShoppingCart;
import vn.edu.iuh.sessioncart.session.UserSession;

import java.io.IOException;
import java.util.Optional;

@WebServlet("/cart/add")
public class CartServlet extends HttpServlet {
    @Inject
    private ProductCatalog productCatalog;

    @Inject
    private UserSession userSession;

    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws IOException {
        if (!userSession.isLoggedIn()) {
            response.sendRedirect(request.getContextPath() +
                "/login");
            return;
        }

        try {
            long productId =
                Long.parseLong(request.getParameter("productId"));

            //productCatalog.findById(productId).ifPresent(userSession.getShoppingCart()::add);

            // Bước 1: Tìm sản phẩm theo ID
            Optional<Product> optionalProduct =
                productCatalog.findById(productId);

            // Bước 2: Kiểm tra sản phẩm có tồn tại hay không
            if (optionalProduct.isPresent()) {
                // Bước 3: Lấy Product ra khỏi Optional
                Product product = optionalProduct.get();

                // Bước 4: Lấy giỏ hàng của session hiện tại
                ShoppingCart shoppingCart =
                    userSession.getShoppingCart();

```

```

        // Bước 5: Thêm sản phẩm vào giỏ hàng
        shoppingCart.add(product);
    }
} catch (NumberFormatException ignored) {
    // ID không hợp lệ: không thay đổi giỏ hàng.
}

// PRG: tránh thêm lại sản phẩm khi người dùng refresh
trang.
response.sendRedirect(request.getContextPath() +
"/products");
}
}

<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Đăng nhập</title>
    <link rel="stylesheet"
href="{pageContext.request.contextPath}/assets/style.css">
</head>
<body>
<main class="card login-card">
    <h1>Đăng nhập</h1>
    <p class="hint">Tài khoản demo: <strong>student</strong> /
<strong>123456</strong></p>
    <p class="error">${error}</p>

    <form method="post"
action="{pageContext.request.contextPath}/login">
        <label for="username">Tên đăng nhập</label>
        <input id="username" name="username" value="{username}"
required autofocus>

        <label for="password">Mật khẩu</label>
        <input id="password" name="password" type="password"
required>

        <button type="submit">Đăng nhập</button>
    </form>
</main>

```

```

</body>
</html>

<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Danh sách sản phẩm</title>
    <link rel="stylesheet"
href="${pageContext.request.contextPath}/assets/style.css">
</head>
<body>
<main class="page">
    <header>
        <div>
            <h1>Danh sách sản phẩm</h1>
            <p>Xin chào,
<strong>${userSession.username}</strong></p>
        </div>
        <form method="post"
action="${pageContext.request.contextPath}/logout">
            <button class="secondary" type="submit">Đăng
xuất</button>
        </form>
    </header>

    <section class="card">
        <table>
            <thead><tr><th>Sân
phẩm</th><th>Giá</th><th></th></tr></thead>
            <tbody>
                <c:forEach items="${products}" var="product">
                    <tr>
                        <td>${product.name}</td>
                        <td><fmt:formatNumber value="${product.price}"
type="currency" currencyCode="USD"/></td>
                        <td>
                            <form method="post"
action="${pageContext.request.contextPath}/cart/add">
                                <input type="hidden" name="productId"
value="${product.id}">

```

```

        <button type="submit">Thêm vào
giỏ</button>
        </form>
    </td>
</tr>
</c:forEach>
</tbody>
</table>
</section>

<section class="card">
    <h2>Giỏ hàng trong session</h2>
    <c:choose>
        <c:when test="${empty userSession.shoppingCart.items}">
            <p>Giỏ hàng đang trống.</p>
        </c:when>
        <c:otherwise>
            <ul>
                <c:forEach
items="${userSession.shoppingCart.items}" var="item">
                    <li>${item.product.name} × ${item.quantity}
                    <fmt:formatNumber
value="${item.subtotal}" type="currency" currencyCode="USD"/>
                    </li>
                </c:forEach>
            </ul>
        </c:otherwise>
    </c:choose>
    <p><strong>Số lượng:</strong>
${userSession.shoppingCart.itemCount}</p>
    <p><strong>Tổng tiền:</strong>
    <fmt:formatNumber
value="${userSession.shoppingCart.total}" type="currency"
currencyCode="USD"/>
    </p>
    <p class="technical">CDI session instance:
${userSession.instanceId}</p>
</section>
</main>
</body>
</html>

```


2. Thực hành: REST API

REST API là cách các thành phần của ứng dụng trao đổi dữ liệu qua HTTP. Trong dự án Shopping Cart:

JSP/JavaScript → HTTP request → Jakarta REST Resource

↓ @Inject
CDI Services

↓
UserSession @SessionScoped

1. REST API là gì?

API là giao diện để hai phần mềm giao tiếp với nhau. REST API là API sử dụng HTTP và biểu diễn dữ liệu thường bằng JSON.

Ví dụ lấy danh sách sản phẩm:

GET /api/products

Response:

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 1500.00
  },
  {
    "id": 2,
    "name": "Keyboard",
    "price": 80.00
  }
]
```

Thêm sản phẩm vào giỏ:

POST /api/cart/items

Content-Type: application/json

```
{
  "productId": 1
}
```

Response:

```
{
  "items": [
```

```
{
  "product": {
    "id": 1,
    "name": "Laptop",
    "price": 1500.00
  },
  "quantity": 1,
  "subtotal": 1500.00
},
"itemCount": 1,
"total": 1500.00
}
```

REST API không trả về giao diện HTML. Nó chủ yếu trả về dữ liệu và HTTP status code.

2. Tại sao cần dùng REST API?

Trong ứng dụng Servlet MVC truyền thống:

Browser → Servlet → request.setAttribute() → JSP → HTML

Servlet thường forward trực tiếp đến JSP:

```
request.setAttribute("products", products);
request.getRequestDispatcher("/WEB-INF/views/products.jsp")
    .forward(request, response);
```

Với REST API:

Browser → REST API → JSON

JSP/JavaScript ← JSON

JavaScript gọi API:

```
const response = await fetch("/api/products");
const products = await response.json();
```

Lợi ích:

- Frontend và backend tách biệt hơn.
- Một API có thể phục vụ JSP, React, Vue, Angular hoặc ứng dụng mobile.
- Dễ kiểm thử bằng Postman, [curl](#) hoặc IntelliJ HTTP Client.
- Dữ liệu JSON dễ truyền giữa các hệ thống.
- Không cần tải lại toàn bộ trang sau mỗi thao tác (giảm tải mạng).
- Backend tập trung vào dữ liệu và nghiệp vụ, Frontend xử lý giao diện.
- HTTP method thể hiện rõ hành động: [GET](#), [POST](#), [PUT](#), [DELETE](#).

3. REST API và RESTful API

Trong Jakarta EE, RESTful API được viết bằng Jakarta RESTful Web Services, trước đây gọi là JAX-RS.

Ví dụ:

```
@Path("/products")
@Produces(MediaType.APPLICATION_JSON)
public class ProductResource {

    @GET
    public List<Product> findAll() {
        return productCatalog.findAll();
    }
}
```

Các annotation quan trọng:

Annotation	Công dụng
<code>@ApplicationPath("/api")</code>	URL gốc của API
<code>@Path("/products")</code>	Đường dẫn resource
<code>@GET</code>	Xử lý HTTP GET
<code>@POST</code>	Xử lý HTTP POST
<code>@PUT</code>	Xử lý HTTP PUT
<code>@DELETE</code>	Xử lý HTTP DELETE
<code>@Produces</code>	Kiểu dữ liệu response
<code>@Consumes</code>	Kiểu dữ liệu request
<code>@PathParam</code>	Đọc giá trị trong URL
<code>@QueryParam</code>	Đọc query parameter

Dưới đây là cách tạo riêng backend `shopping-cart-api` bằng template **Jakarta EE REST Service** trong IntelliJ IDEA Ultimate, chạy trên Tomcat 11, trả JSON và sử dụng CDI/Weld.

IntelliJ có thể thay đổi nhẹ tên nút theo phiên bản, nhưng luồng chung là **New Project** → **Jakarta EE** → **REST Service**. IntelliJ Ultimate tích hợp sẵn hỗ trợ Jakarta REST/JAX-RS. [Hướng dẫn chính thức của JetBrains](#)

Dự án Shopping Cart

BackEnd: Jakarta EE: RESTful API

1. Kiểm tra plugin IntelliJ

Mở:

IntelliJ IDEA → Settings → Plugins → Installed

Đảm bảo những plugin sau đang bật:

- Jakarta EE Platform
- Jakarta EE: Application Servers
- Jakarta EE: Web/Servlets
- Jakarta EE: RESTful Web Services (JAX-RS)
- Tomcat and TomEE

Nếu vừa bật plugin, khởi động lại IntelliJ.

2. Chuẩn bị JDK và Tomcat

Cần:

JDK: Java 21

Server: Tomcat 11

Đăng ký Tomcat trong IntelliJ:

1. Mở **Settings**.
2. Chọn **Build, Execution, Deployment** → **Application Servers**.
3. Bấm **+**.
4. Chọn **Tomcat Server**.
5. Chọn thư mục cài Tomcat 11.
6. Bấm **OK**.

3. Tạo Jakarta EE REST Service project

Trong IntelliJ:

1. Chọn **File** → **New** → **Project**.
2. Bên trái chọn **Jakarta EE**.
3. Nhập:

Name: shopping-cart-api

4. Template: REST Service
5. Language: Java
6. Build system: Maven
7. JDK: 21
8. Application server có thể để trống rồi cấu hình sau, hoặc chọn Tomcat 11 vừa đăng ký.
9. Group: vn.edu.iuh
10. Bấm **Next**.

4. Chọn Jakarta EE version và dependencies

Đối với bộ mã Shopping Cart đã xây dựng, chọn:

Jakarta EE 10

Tomcat 11 vẫn chạy được ứng dụng dùng Servlet 6.0/Jakarta EE 10 API. Việc chọn EE 10 giúp đồng bộ với:

CDI 4.0

Weld 5

Jakarta REST 3.1

Jersey 3.1

Trong danh sách dependencies, chọn:

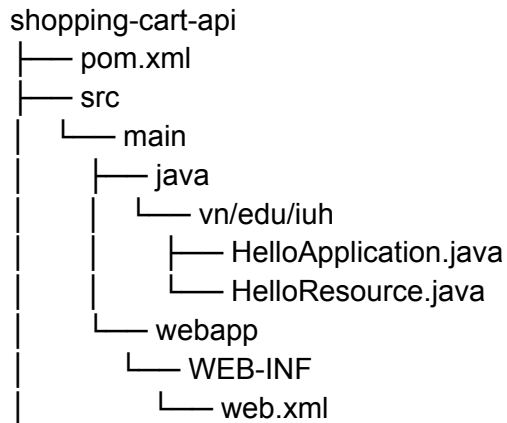
- Contexts and Dependency Injection — CDI
- RESTful Web Services — JAX-RS
- Servlet
- Eclipse Jersey Server
- Jersey JSON Jackson nếu có
- Weld

Một số phiên bản IntelliJ hiển thị **Weld SE**. Bạn vẫn có thể chọn để wizard tạo cấu trúc, nhưng sau đó sẽ thay dependency bằng **weld-servlet-core**, vì ứng dụng chạy trong Tomcat chứ không phải Java SE độc lập.

Bấm **Create**.

5. Kiểm tra cấu trúc được tạo

IntelliJ thường tạo cấu trúc gần giống:



Wizard có thể tạo các class tên:

HelloApplication
HelloResource

Có thể xóa chúng sau khi tạo các class Shopping Cart tương ứng.

6. Chỉnh thông tin Maven (file pom.xml)

7. Khai báo dependency cho Tomcat(file pom.xml)

Nội dung file pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>vn.edu.iuh</groupId>
    <artifactId>shopping-cart-api</artifactId>
    <version>1.0-SNAPSHOT</version>
    <name>shopping-cart-api</name>
    <packaging>war</packaging>

    <properties>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.target>21</maven.compiler.target>
        <maven.compiler.source>21</maven.compiler.source>
        <junit.version>5.10.2</junit.version>
    </properties>

    <dependencies>

```

```

<dependency>
  <groupId>jakarta.servlet</groupId>
  <artifactId>jakarta.servlet-api</artifactId>
  <version>6.1.0</version>
  <scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-servlet</artifactId>
  <version>3.1.6</version>
</dependency>

<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-json-jackson</artifactId>
  <version>3.1.6</version>
</dependency>

<dependency>
  <groupId>org.glassfish.jersey.inject</groupId>
  <artifactId>jersey-hk2</artifactId>
  <version>3.1.6</version>
</dependency>

<dependency>
  <groupId>org.glassfish.jersey.ext.cdi</groupId>
  <artifactId>jersey-cdi1x</artifactId>
  <version>3.1.6</version>
</dependency>

<dependency>
  <groupId>org.jboss.weld.servlet</groupId>
  <artifactId>weld-servlet-core</artifactId>
  <version>5.1.2.Final</version>
</dependency>

<dependency>
  <groupId>com.fasterxml.jackson.module</groupId>
  <artifactId>jackson-module-jaxb-annotations</artifactId>
  <version>2.17.0</version>
</dependency>

<dependency>
  <groupId>javax.xml.bind</groupId>

```

```

        <artifactId>jaxb-api</artifactId>
        <version>2.3.1</version>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-war-plugin</artifactId>
            <version>3.4.0</version>
        </plugin>
    </plugins>
</build>
</project>

```

Bấm nút **Load Maven Changes** ở góc phải **pom.xml**.

8. Tạo các package

Nhấp phải vào:

src/main/java

Chọn:

New → Package

Tạo lần lượt:

vn.edu.iuh.cartapi.api
 vn.edu.iuh.cartapi.model
 vn.edu.iuh.cartapi.service
 vn.edu.iuh.cartapi.session

Kết quả:

```

vn.edu.iuh.cartapi
├── api
├── model
├── service
└── session

```

9. Kích hoạt CDI bằng **beans.xml**

Nhấp phải:

src/main/webapp/WEB-INF

Chọn:

New → File

Tên:

beans.xml

Nội dung:

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    https://jakarta.ee/xml/ns/jakartaee
    https://jakarta.ee/xml/ns/jakartaee/beans_4_0.xsd"
  version="4.0"
  bean-discovery-mode="annotated"/>
```

Vị trí phải chính xác:

src/main/webapp/WEB-INF/beans.xml

10. Tạo **web.xml**

Trong **WEB-INF**, tạo hoặc sửa **web.xml**:

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
  version="6.0">

  <display-name>Shopping Cart REST API</display-name>

  <session-config>
    <session-timeout>20</session-timeout>
  </session-config>
</web-app>
```

11. Tạo Jakarta REST Application

Trong package:

vn.edu.iuh.cartapi.api

Tạo class **RestApplication**:

```
package vn.edu.iuh.cartapi.api;

import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;

@ApplicationPath("/api")
public class RestApplication extends Application {
}
```

Ý nghĩa:

Context path: /shopping-cart-api

API path: /api

Do đó URL gốc là:

http://localhost:8080/shopping-cart-api/api

12. Tạo model **Product**

Trong package **model**, tạo **Product**:

```
package vn.edu.iuh.cartapi.model;

import java.io.Serializable;
import java.io.Serializable;
import java.math.BigDecimal;

public class Product implements Serializable {

    @Serial
    private static final long serialVersionUID = 1L;

    private final long id;
    private final String name;
    private final BigDecimal price;

    public Product(
        long id,
        String name,
        BigDecimal price
    ) {
        this.id = id;
    }
}
```

```

        this.name = name;
        this.price = price;
    }

    public long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public BigDecimal getPrice() {
        return price;
    }
}

```

Jackson dùng các getter để tạo JSON:

```

{
  "id": 1,
  "name": "Laptop",
  "price": 1500.00
}

```

13. Tạo model **CartItem**

```

package vn.edu.iuh.cartapi.model;

import java.io.Serializable;
import java.io.Serializable;
import java.math.BigDecimal;

public class CartItem implements Serializable {

    @Serial
    private static final long serialVersionUID = 1L;

    private final Product product;
    private int quantity = 1;

    public CartItem(Product product) {
        this.product = product;
    }

    public void increaseQuantity() {
        quantity++;
    }
}

```

```

    }

    public Product getProduct() {
        return product;
    }

    public int getQuantity() {
        return quantity;
    }

    public BigDecimal getSubtotal() {
        return product.getPrice()
            .multiply(BigDecimal.valueOf(quantity));
    }
}

```

14. Tạo CDI service **ProductCatalog**

Trong package **service**:

```

package vn.edu.iuh.cartapi.service;

import jakarta.enterprise.context.ApplicationScoped;
import vn.edu.iuh.cartapi.model.Product;

import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;

@ApplicationScoped
public class ProductCatalog {

    private final List<Product> products = List.of(
        new Product(
            1,
            "Laptop",
            new BigDecimal("1500.00")
        ),
        new Product(
            2,
            "Keyboard",
            new BigDecimal("80.00")
        ),
        new Product(
            3,
            "Mouse",

```

```

        new BigDecimal("40.00")
    )
);

public List<Product> findAll() {
    return products;
}

public Optional<Product> findById(long id) {
    return products.stream()
        .filter(product ->
            product.getId() == id
        )
        .findFirst();
}
}

```

`@ApplicationScoped` nghĩa là CDI tạo một `ProductCatalog` dùng chung cho toàn ứng dụng.

15. Tạo `AuthenticationService`

```

package vn.edu.iuh.cartapi.service;

import jakarta.enterprise.context.ApplicationScoped;

@ApplicationScoped
public class AuthenticationService {

    public boolean authenticate(
        String username,
        String password
    ){
        return "student".equals(username)
            && "123456".equals(password);
    }
}

```

16. Tạo `ShoppingCart`

```

package vn.edu.iuh.cartapi.service;

import vn.edu.iuh.cartapi.model.CartItem;
import vn.edu.iuh.cartapi.model.Product;

import java.io.Serial;

```

```

import java.io.Serializable;
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class ShoppingCart implements Serializable {

    @Serial
    private static final long serialVersionUID = 1L;

    private final List<CartItem> items =
        new ArrayList<>();

    public void add(Product product) {
        for (CartItem item : items) {
            if (item.getProduct().getId()
                == product.getId()) {
                item.increaseQuantity();
                return;
            }
        }

        items.add(new CartItem(product));
    }

    public List<CartItem> getItems() {
        return Collections.unmodifiableList(items);
    }

    public int getItemCount() {
        return items.stream()
            .mapToInt(CartItem::getQuantity)
            .sum();
    }

    public BigDecimal getTotal() {
        return items.stream()
            .map(CartItem::getSubtotal)
            .reduce(
                BigDecimal.ZERO,
                BigDecimal::add
            );
    }
}

```

17. Tạo `UserSession` `@SessionScoped`

Trong package `session`:

```
package vn.edu.iuh.cartapi.session;

import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import jakarta.enterprise.context.SessionScoped;
import vn.edu.iuh.cartapi.service.ShoppingCart;

import java.io.Serial;
import java.io.Serializable;
import java.util.UUID;

@SessionScoped
public class UserSession implements Serializable {

    @Serial
    private static final long serialVersionUID = 1L;

    private final String instanceId =
        UUID.randomUUID().toString();

    private final ShoppingCart shoppingCart =
        new ShoppingCart();

    private String username;

    @PostConstruct
    public void created() {
        System.out.println(
            "CREATED UserSession: " + instanceId
        );
    }

    @PreDestroy
    public void destroyed() {
        System.out.println(
            "DESTROYED UserSession: " + instanceId
        );
    }

    public void login(String username) {
        this.username = username;
    }
}
```

```

public boolean isLoggedIn() {
    return username != null;
}

public String getUsername() {
    return username;
}

public String getInstanceId() {
    return instanceId;
}

public ShoppingCart getShoppingCart() {
    return shoppingCart;
}
}

```

18. Tạo AuthResource

Trong package `api`, tạo:

```

@Path("/auth")
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
@RequestScoped
public class AuthResource {

    @Inject
    AuthenticationService authenticationService;

    @Inject
    UserSession userSession;

    @POST
    @Path("/login")
    public Response login(LoginRequest body) {
        if (body == null
            || !authenticationService.authenticate(
                body.username(),
                body.password()
            )) {

            return Response.status(
                Response.Status.UNAUTHORIZED
            )
                .entity(Map.of(

```



```

        "message",
        "Sai tài khoản hoặc mật khẩu"
    ))
    .build();
}

userSession.login(body.username());

return Response.ok(Map.of(
    "loggedIn", true,
    "username", userSession.getUsername(),
    "instanceId",
    userSession.getInstanceId()
)).build();
}

public record LoginRequest(
    String username,
    String password
){
}
}

```

Thêm API kiểm tra session:

```

@GET
@Path("/session")
public Map<String, Object> session() {
    return Map.of(
        "loggedIn", userSession.isLoggedIn(),
        "username",
        userSession.getUsername() == null
        ? ""
        : userSession.getUsername(),
        "instanceId",
        userSession.getInstanceId()
    );
}

```

Thêm logout:

```

@DELETE
@Path("/session")
public Response logout(
    @Context HttpServletRequest request
){
    if (request.getSession(false) != null) {
        request.getSession(false).invalidate();
    }
}

```

```

    }

    return Response.noContent().build();
}

```

Class đầy đủ có tại

[AuthResource.java](/Users/nguyenvulam/.codex/.chatgpt-projects/g-p-69548b1ecd70819180d6276470ca6952/shopping-cart-api/src/main/java/vn/edu/iuh/cartapi/api/AuthResource.java).

19. Tạo ProductResource

```

@Path("/products")
@Produces(MediaType.APPLICATION_JSON)
@RequestScoped
public class ProductResource {

    @Inject
    ProductCatalog productCatalog;

    @Inject
    UserSession userSession;

    @GET
    public Response findAll() {
        if (!userSession.isLoggedIn()) {
            return Response.status(401)
                .entity(Map.of(
                    "message",
                    "Bạn chưa đăng nhập"
                ))
                .build();
        }

        return Response.ok(
            productCatalog.findAll()
        ).build();
    }
}

```

20. Tạo CartResource

```

@Path("/cart")
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
@RequestScoped

```

```

public class CartResource {

    @Inject
    ProductCatalog productCatalog;

    @Inject
    UserSession userSession;

    @GET
    public Response getCart() {
        if (!userSession.isLoggedIn()) {
            return unauthorized();
        }

        return Response.ok(cartBody()).build();
    }

    @POST
    @Path("/items")
    public Response addItem(
        AddItemRequest request
    ) {
        if (!userSession.isLoggedIn()) {
            return unauthorized();
        }

        Optional<Product> product =
            request == null
                ? Optional.empty()
                : productCatalog.findById(
                    request.productId()
                );

        if (product.isEmpty()) {
            return Response.status(404)
                .entity(Map.of(
                    "message",
                    "Không tìm thấy sản phẩm"
                ))
                .build();
        }

        userSession.getShoppingCart()
            .add(product.get());

        return Response.ok(cartBody()).build();
    }
}

```

```

private Map<String, Object> cartBody() {
    ShoppingCart cart =
        userSession.getShoppingCart();

    return Map.of(
        "items", cart.getItems(),
        "itemCount", cart.getItemCount(),
        "total", cart.getTotal()
    );
}

private Response unauthorized() {
    return Response.status(401)
        .entity(Map.of(
            "message",
            "Bạn chưa đăng nhập"
        ))
        .build();
}

public record AddItemRequest(long productId) {
}
}

```

21. Tạo Tomcat configuration

Chọn:

Run → Edit Configurations

Sau đó:

1. Bấm **+**.
2. Chọn **Tomcat Server** → **Local**.
3. Name:

Shopping Cart API

4. Application server: Tomcat 11.
5. HTTP port: **8080**.
6. Mở tab **Deployment**.
7. Bấm **+**.
8. Chọn:

shopping-cart-api:war exploded

9. Application context:

/shopping-cart-api

10. Bấm **Apply** → **OK**.

22. Build và chạy

Chọn:

Build → Rebuild Project

Sau đó chạy cấu hình **Shopping Cart API**.

Trong cửa sổ Run phải có thông báo tương tự:

Artifact shopping-cart-api:war exploded:
Artifact is deployed successfully

23. Tạo IntelliJ HTTP Client file

Nhấp phải vào thư mục gốc project:

New → HTTP Request

Đặt tên:

shopping-cart.http

Nội dung:

@baseUrl = http://localhost:8080/shopping-cart-api/api

Login

POST {{baseUrl}}/auth/login

Content-Type: application/json

```
{
  "username": "student",
  "password": "123456"
}
```

Kiểm tra session

GET {{baseUrl}}/auth/session

Danh sách sản phẩm

GET {{baseUrl}}/products

Thêm Laptop

POST {{baseUrl}}/cart/items
Content-Type: application/json

```
{
  "productId": 1
}
```

Xem giỏ hàng
GET {{baseUrl}}/cart

Logout
DELETE {{baseUrl}}/auth/session

Bấm biểu tượng chạy cạnh từng request theo thứ tự.

IntelliJ HTTP Client lưu cookie **JSESSIONID**, nên các request sau login thuộc cùng một HTTP session.

24. Kết quả mong đợi

Login:

```
{
  "loggedIn": true,
  "username": "student",
  "instanceId": "..."
}
```

Products:

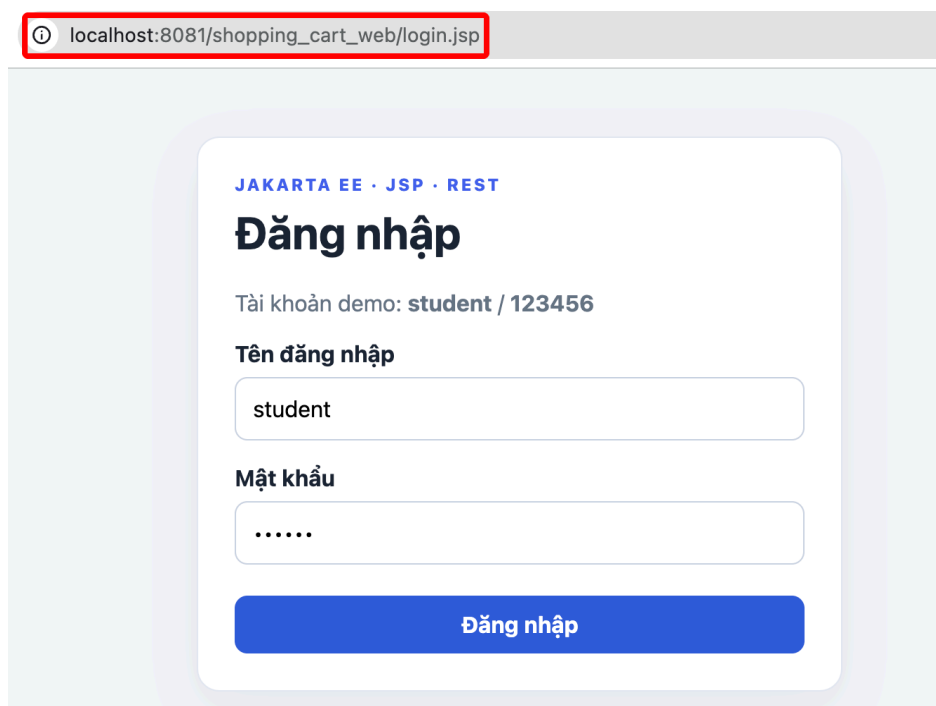
```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 1500.00
  },
  {
    "id": 2,
    "name": "Keyboard",
    "price": 80.00
  },
  {
    "id": 3,
    "name": "Mouse",
    "price": 40.00
  }
]
```

Cart:

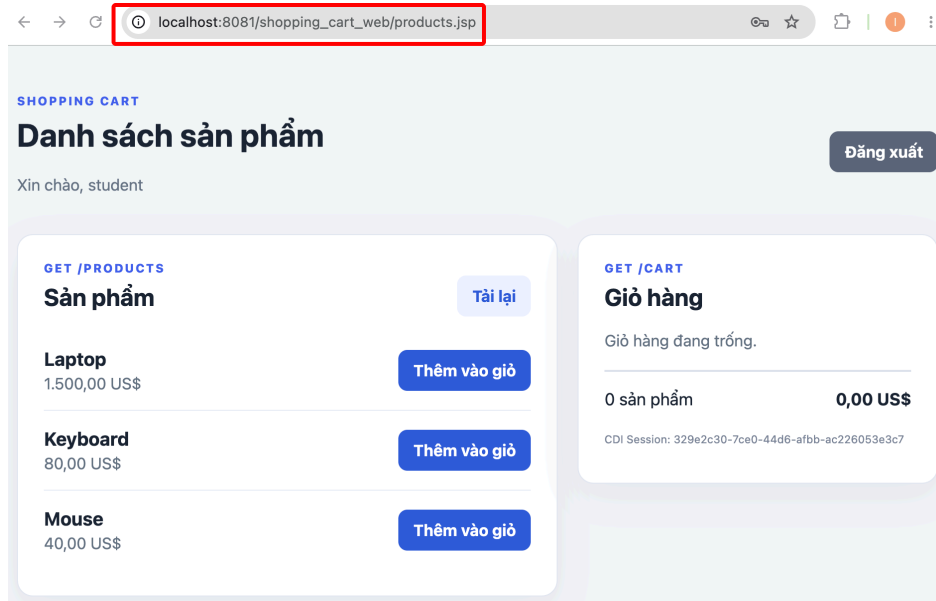
```
{  
  "items": [],  
  "itemCount": 0,  
  "total": 0  
}
```

FrontEnd: WEB app (các trang JSP gọi BackEnd: các REST APIs)

Tạo dự án Jakarta EE: Web app



Hình 3- Màn hình đăng nhập login.jsp



Hình 4- Màn hình products.jsp

Tạo 3 file: style.css

```
:root {
  font-family: Inter, ui-sans-serif, system-ui, -apple-system,
  sans-serif;
  color: #1c2434;
  background: #f3f5f9;
}
* { box-sizing: border-box; }
body { margin: 0; }
.page, .login-card { width: min(1080px, calc(100% - 32px));
margin: 48px auto; }
.login-card { max-width: 460px; }
.card { background: #fff; padding: 26px; border: 1px solid
#e3e8f2; border-radius: 16px; box-shadow: 0 12px 35px #18233d10; }
.eyebrow { margin: 0 0 8px; color: #4263eb; font-size: 12px;
font-weight: 800; letter-spacing: .13em; }
h1, h2, h3 { margin-top: 0; }
.muted { color: #687386; }
label { display: block; margin: 17px 0 7px; font-weight: 700; }
input { width: 100%; padding: 12px; border: 1px solid #cbd3e2;
border-radius: 9px; font: inherit; }
button, .button { border: 0; border-radius: 9px; padding: 11px
15px; color: #fff; background: #315bd7; font: inherit;
font-weight: 700; cursor: pointer; }
button:disabled { opacity: .55; cursor: wait; }
.login-card button { width: 100%; margin-top: 22px; }
.secondary { background: #596579; }
.subtle { color: #315bd7; background: #edf2ff; }
```



```

.topbar, .section-heading, .product, .cart-list li, .cart-summary
{ display: flex; align-items: center; justify-content:
space-between; gap: 18px; }
.topbar { margin-bottom: 24px; }
.layout { display: grid; grid-template-columns: 1.5fr 1fr; gap:
20px; align-items: start; }
.product { padding: 17px 0; border-bottom: 1px solid #e9edf4; }
.product:last-child { border-bottom: 0; }
.product h3, .product p { margin: 0; }
.price { color: #687386; margin-top: 4px !important; }
.cart-list { padding: 0; margin: 0; list-style: none; }
.cart-list li { padding: 12px 0; border-bottom: 1px solid #e9edf4;
}
.cart-summary { margin-top: 20px; padding-top: 17px; border-top:
2px solid #dce3ef; font-size: 17px; }
.message { padding: 12px 15px; margin-bottom: 18px; border-radius:
9px; }
.error { color: #912018; background: #feeceb; }
.success { color: #176b3a; background: #e8f8ee; }
.technical { color: #7a8497; margin-top: 24px; font-size: 11px;
overflow-wrap: anywhere; }
@media (max-width: 760px) { .layout { grid-template-columns: 1fr;
} .topbar { align-items: flex-start; } }

```

Login.jsp

```

<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1">
  <title>Đăng nhập Shopping Cart</title>
  <link rel="stylesheet"
href="${pageContext.request.contextPath}/assets/style.css">
</head>
<body>
<main class="card login-card">
  <p class="eyebrow">JAKARTA EE · JSP · REST</p>
  <h1>Đăng nhập</h1>
  <p class="muted">Tài khoản demo: <strong>student</strong> /
<strong>123456</strong></p>
  <div id="error" class="message error" hidden></div>

  <form id="loginForm">

```

```

    <label for="username">Tên đăng nhập</label>
    <input id="username" name="username" value="student"
autocomplete="username" required autofocus>

    <label for="password">Mật khẩu</label>
    <input id="password" name="password" type="password"
value="123456"
        autocomplete="current-password" required>

    <button id="loginButton" type="submit">Đăng nhập</button>
  </form>
</main>

<script>
  const API = '${initParam.apiUrl}';
  const contextPath = '${pageContext.request.contextPath}';
  const form = document.querySelector('#loginForm');
  const errorBox = document.querySelector('#error');
  const button = document.querySelector('#loginButton');

  // Nếu backend vẫn còn session đăng nhập, đi thẳng đến trang
  sản phẩm.
  fetch(API + '/auth/session', {credentials: 'include'})
    .then(response => response.ok ? response.json() : null)
    .then(session => {
      if (session?.loggedIn) location.href = contextPath +
'/products.jsp';
    })
    .catch(() => {});

  form.addEventListener('submit', async event => {
    event.preventDefault();
    errorBox.hidden = true;
    button.disabled = true;
    button.textContent = 'Đang đăng nhập...';

    try {
      const response = await fetch(API + '/auth/login', {
        method: 'POST',
        credentials: 'include',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({
          username:
document.querySelector('#username').value,
          password:
document.querySelector('#password').value

```

```

        })
    });

    const data = await response.json();
    if (!response.ok) throw new Error(data.message || 'Đăng
nhập thất bại');
    location.href = contextPath + '/products.jsp';
} catch (error) {
    errorBox.textContent = error.message === 'Failed to
fetch'
        ? 'Không kết nối được backend REST API.' :
error.message;
    errorBox.hidden = false;
} finally {
    button.disabled = false;
    button.textContent = 'Đăng nhập';
}
});
</script>
</body>
</html>

```

Products.jsp

```

<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Shopping Cart</title>
    <link rel="stylesheet"
href="${pageContext.request.contextPath}/assets/style.css">
</head>
<body>
<main class="page">
    <header class="topbar">
        <div>
            <p class="eyebrow">SHOPPING CART</p>
            <h1>Danh sách sản phẩm</h1>
            <p id="welcome" class="muted">Đang tải session...</p>
        </div>
        <button id="logoutButton" class="button secondary"
type="button">Đăng xuất</button>

```

```

</header>

<div id="message" class="message" hidden></div>

<div class="layout">
  <section class="card">
    <div class="section-heading">
      <div><p class="eyebrow">GET /PRODUCTS</p><h2>Sản
phẩm</h2></div>
      <button id="reloadButton" class="button subtle"
type="button">Tải lại</button>
    </div>
    <div id="products" class="product-list"><p
class="muted">Đang tải sản phẩm...</p></div>
  </section>

  <aside class="card cart-card">
    <p class="eyebrow">GET /CART</p>
    <h2>Giỏ hàng</h2>
    <div id="cart"><p class="muted">Đang tải giỏ
hàng...</p></div>
    <p id="instanceId" class="technical"></p>
  </aside>
</div>
</main>

<script>
  const API = '${initParam.apiUrl}';
  const contextPath = '${pageContext.request.contextPath}';
  const productBox = document.querySelector('#products');
  const cartBox = document.querySelector('#cart');
  const messageBox = document.querySelector('#message');
  const money = value => new Intl.NumberFormat('vi-VN', {
    style: 'currency', currency: 'USD'
  }).format(value);

  async function callApi(path, options = {}) {
    const response = await fetch(API + path, {...options,
credentials: 'include'});
    if (response.status === 401) {
      location.href = contextPath + '/login.jsp';
      throw new Error('Session đăng nhập đã hết hạn');
    }
    const data = response.status === 204 ? null : await
response.json();

```

```

        if (!response.ok) throw new Error(data?.message || 'REST
API trả lỗi ' + response.status);
        return data;
    }

    function showMessage(text, error = false) {
        messageBox.textContent = text;
        messageBox.className = error ? 'message error' : 'message
success';
        messageBox.hidden = false;
        setTimeout(() => messageBox.hidden = true, 2500);
    }

    function renderProducts(products) {
        productBox.replaceChildren();
        for (const product of products) {
            const row = document.createElement('article');
            row.className = 'product';

            const description = document.createElement('div');
            const name = document.createElement('h3');
            name.textContent = product.name;
            const price = document.createElement('p');
            price.className = 'price';
            price.textContent = money(product.price);
            description.append(name, price);

            const addButton = document.createElement('button');
            addButton.className = 'button';
            addButton.textContent = 'Thêm vào giỏ';
            addButton.addEventListener('click', () =>
addToCart(product.id, addButton));
            row.append(description, addButton);
            productBox.append(row);
        }
    }

    function renderCart(cart) {
        cartBox.replaceChildren();
        if (cart.items.length === 0) {
            const empty = document.createElement('p');
            empty.className = 'muted';
            empty.textContent = 'Giỏ hàng đang trống.';
            cartBox.append(empty);
        } else {
            const list = document.createElement('ul');

```

```

    list.className = 'cart-list';
    for (const item of cart.items) {
        const row = document.createElement('li');
        const label = document.createElement('span');
        label.textContent = item.product.name + ' x ' +
item.quantity;
        const subtotal = document.createElement('strong');
        subtotal.textContent = money(item.subtotal);
        row.append(label, subtotal);
        list.append(row);
    }
    cartBox.append(list);
}

const summary = document.createElement('div');
summary.className = 'cart-summary';
summary.innerHTML = '<span>' + cart.itemCount + ' sản
phẩm</span><strong>'
    + money(cart.total) + '</strong>';
cartBox.append(summary);
}

```

```

async function addToCart(productId, button) {
    button.disabled = true;
    try {
        const cart = await callApi('/cart/items', {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({productId})
        });
        renderCart(cart);
        showMessage('Đã thêm sản phẩm vào giỏ hàng.');
```

```

    } catch (error) {
        showMessage(error.message, true);
    } finally {
        button.disabled = false;
    }
}

```

```

async function loadPage() {
    try {
        const [session, products, cart] = await Promise.all([
            callApi('/auth/session'),
            callApi('/products'),
            callApi('/cart')
        ]);
    }
}

```

```

        if (!session.loggedIn) {
            location.href = contextPath + '/login.jsp';
            return;
        }
        document.querySelector('#welcome').textContent = 'Xin
chào, ' + session.username;
        document.querySelector('#instanceId').textContent =
'CDI Session: ' + session.instanceId;
        renderProducts(products);
        renderCart(cart);
    } catch (error) {
        showMessage(error.message, true);
    }
}

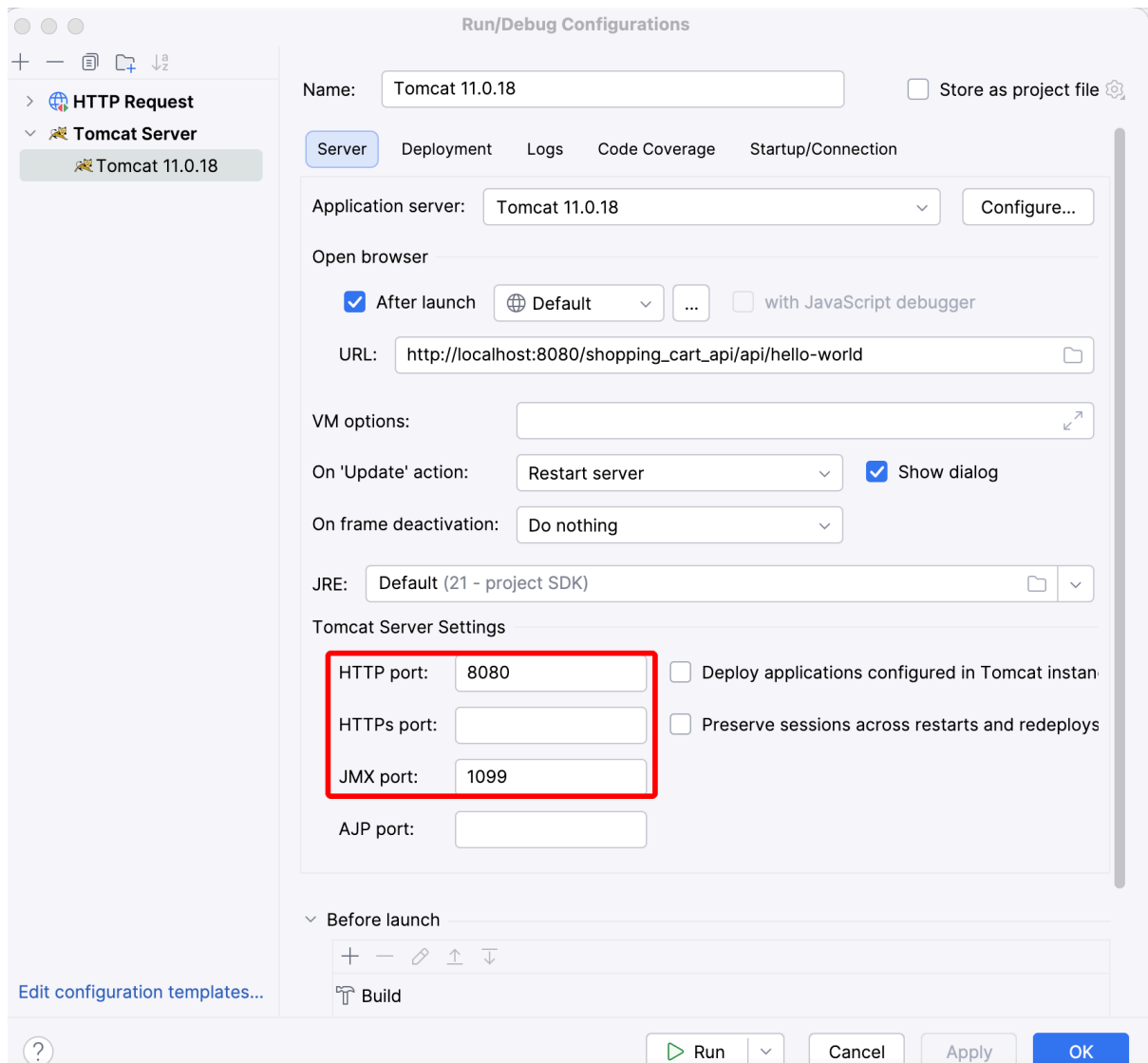
document.querySelector('#reloadButton').addEventListener('click',
loadPage);

document.querySelector('#logoutButton').addEventListener('click',
async () => {
    try {
        await callApi('/auth/session', {method: 'DELETE'});
        location.href = contextPath + '/login.jsp';
    } catch (error) {
        showMessage(error.message, true);
    }
});

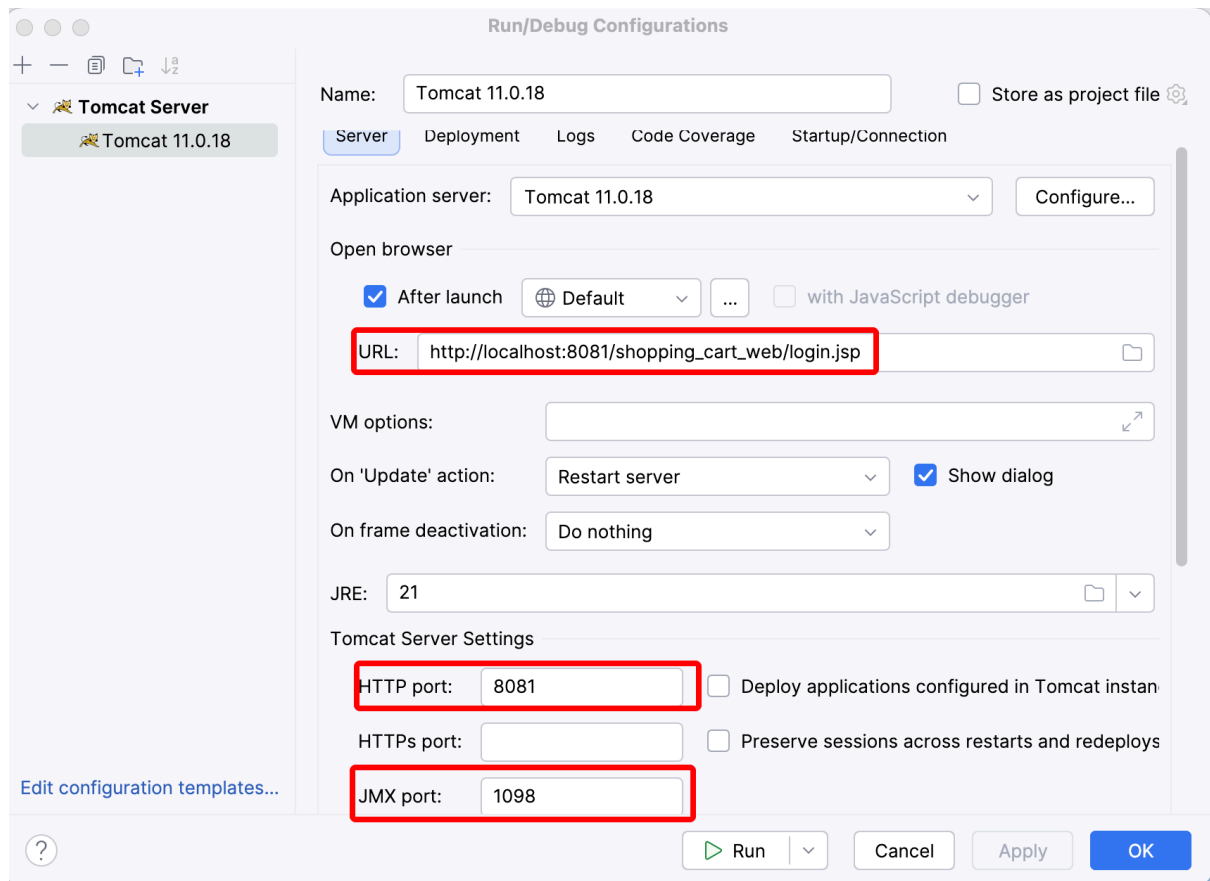
loadPage();
</script>
</body>
</html>

```

Cấu hình để Front End các trang jsp gọi các REST APIs trên Back End



Hình 5- Cấu hình dự án BackEnd chạy (trước) trên cổng HTTP port 8080; JMX port 1099



Hình 6- Cấu hình dự án FrontEnd chạy (sau) trên cổng HTTP port 8081; JMX port 1098